



**SCHOOL OF
COMPUTING**

DESIGN AND ANALYSIS OF ALGORITHMS LAB

WORKBOOK

WEEK - 6

NAME : Leena K

ROLL NUMBER : CH.SC.U4CSE24122

CLASS : CSE-B

Question 1: Write a program to perform Quick Sort by taking First Element, Last Element and a Random Element as Pivot Element for the given numbers:

157, 110, 147, 122, 149, 151, 111, 141, 112, 123, 133, 117

CODE:

```
//CH.SC.U4CSE24122
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
    int height;
};

int max(int a, int b) {
    return (a > b) ? a : b;
}

int height(struct Node *n) {
    if (n == NULL)
        return 0;
    return n->height;
}

struct Node* newNode(int data) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = data;
    node->left = node->right = NULL;
    node->height = 1;
    return node;
}

struct Node* rightRotate(struct Node* y) {
    struct Node* x = y->left;
    struct Node* T2 = x->right;

    x->right = y;
    y->left = T2;

    y->height = max(height(y->left), height(y->right)) + 1;
    x->height = max(height(x->left), height(x->right)) + 1;

    return x;
}

struct Node* leftRotate(struct Node* x) {
    struct Node* y = x->right;
    struct Node* T2 = y->left;

    y->left = x;
    x->right = T2;

    x->height = max(height(x->left), height(x->right)) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;

    return y;
}
```

```

}

int getBalance(struct Node* n) {
    if (n == NULL)
        return 0;
    return height(n->left) - height(n->right);
}

struct Node* avlInsert(struct Node* node, int data) {
    if (node == NULL)
        return newNode(data);

    if (data < node->data)
        node->left = avlInsert(node->left, data);
    else if (data > node->data)
        node->right = avlInsert(node->right, data);
    else
        return node;

    node->height = 1 + max(height(node->left), height(node->right));

    int balance = getBalance(node);

    // Left Left Case
    if (balance > 1 && data < node->left->data)
        return rightRotate(node);

    // Right Right Case
    if (balance < -1 && data > node->right->data)
        return leftRotate(node);

    // Left Right Case
    if (balance > 1 && data > node->left->data) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }

    // Right Left Case
    if (balance < -1 && data < node->right->data) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }

    return node;
}

void levelOrder(struct Node* root) {
    if (root == NULL) {
        printf("Tree is empty\n");
        return;
    }

    struct Node* queue[100];
    int front = 0, rear = 0;
    int currentLevel = 0;

    queue[rear++] = root;

    printf("\nLevel Order Traversal:\n");

```

```

while (front < rear) {
    int levelSize = rear - front;
    printf("Level %d: ", currentLevel++);

    int i;
    for (i = 0; i < levelSize; i++) {
        struct Node* current = queue[front++];
        printf("%d ", current->data);

        if (current->left != NULL)
            queue[rear++] = current->left;
        if (current->right != NULL)
            queue[rear++] = current->right;
    }
    printf("\n");
}

void printTree(struct Node* root, int space) {
    if (root == NULL)
        return;

    space += 10;

    printTree(root->right, space);

    printf("\n");

    int i;
    for (i = 10; i < space; i++)
        printf(" ");

    printf("%d\n", root->data);

    printTree(root->left, space);
}

int main() {
    printf("CH.SC.U4CSE24122\n");

    struct Node* root = NULL;

    int values[] = {157, 110, 147, 122, 149, 151, 111, 141, 112, 123, 133, 117};
    int n = sizeof(values) / sizeof(values[0]);

    printf("BUILDING AVL TREE\n");
    printf("Inserting values: ");

    int i;
    for (i = 0; i < n; i++) {
        printf("%d ", values[i]);
        root = avlInsert(root, values[i]);
    }

    printf("\n");

    levelOrder(root);

```

```
    if (getBalance(root) >= -1 && getBalance(root) <= 1)
        printf("Tree is AVL balanced\n");
    else
        printf("Tree is NOT balanced\n");

    return 0;
}
```

OUTPUT:

```
CH.SC.U4CSE24122
BUILDING AVL TREE
Inserting values: 157 110 147 122 149 151 111 141 112 123 133 117

Level Order Traversal:
Level 0: 122
Level 1: 111 147
Level 2: 110 112 133 151
Level 3: 117 123 141 149 157
Tree is AVL balanced

-----
Process exited after 0.1027 seconds with return value 0
Press any key to continue . . .
```

OUTPUT:

Time Complexity: $O(N^2)$: It takes $O(N^2)$ time in the worst case because each partition processes all elements while reducing the problem size by only one element at a time.

Space Complexity: $O(N)$: $O(N)$ space in the worst case because recursive calls can go as deep as the number of elements when partitions are unbalanced.